

Reddit Clone: Backend API Documentation

A REST API built with Express and MongoDB (Mongoose) powering the Reddit Clone project: auth, communities, posts, comments, and voting.

- **Base URL:** `https://reddit-backend-app.onrender.com`
- **Content type:** `application/json`, except for endpoints that take a file upload, which use `multipart/form-data`
- **Auth:** JWT Bearer token in the Authorization header

Table of Contents

1. [Auth](#)
2. [Users](#)
3. [Communities](#)
4. [Posts](#)
5. [Comments](#)
6. [Votes](#)
7. [File Uploads](#)
8. [Response Shape and Error Handling](#)
9. [Known Issues](#)

Auth

POST /api/auth/register

Create a new account.

Body

```
{
  "username": "string",
  "email": "string",
  "password": "string"
}
```

201 Response

```
{
  "message": "User registered successfully",
  "data": {
    "userId": "USER_XXXXXX",
    "username": "string",
    "email": "string",
    "avatar": null,
    "token": "eyJhbGciOi..."
  }
}
```

400: username or email already exists.

POST /api/auth/login

```
{ "email": "string", "password": "string" }
```

200 Response

```

{
  "success": true,
  "message": "Login successful",
  "data": {
    "userId": "USER_XXXXXX",
    "username": "string",
    "email": "string",
    "avatar": null,
    "createdAt": "2026-01-01T00:00:00.000Z",
    "token": "eyJhbGciOi..."
  }
}

```

401: invalid email or password.

Store token (in memory or localStorage) and send it as Authorization: Bearer <token> on every protected request below.

Users

GET /api/users/:username

Public. Fetch a user's profile.

200 returns { success: true, message, data: <user> } **404:** user not found.

GET /api/users

Public. Returns all users. There is no pagination currently.

200 returns { message, totalUsers, data: [<user>, ...] } **404:** no users found.

PUT /api/users/me (protected)

Update the logged-in user's profile. Send only the fields you want to change; omitted fields are left as is.

Body (all optional, send at least one)

```

{
  "bio": "string",
  "username": "string"
}

```

200 Response

```

{
  "success": true,
  "message": "Profile updated",
  "data": { "...": "full updated user object" }
}

```

401: not authenticated. **404:** user not found.

If you change username to one that's already taken, expect a 500. The schema enforces uniqueness but this route doesn't pre-check it, so either surface the raw error message to the user or add a friendlier check on the frontend.

POST /api/users/me/avatar (protected)

Upload or replace the logged-in user's avatar.

Body: multipart/form-data, field name image.

200 Response

```
{
  "success": true,
  "message": "Profile picture updated successfully",
  "data": { "...": "updated user object" }
}
```

400: no file provided. **401:** not authenticated.

User object shape (passwordHash is stripped automatically):

```
{
  "userId": "USER_XXXXXX",
  "username": "string",
  "email": "string",
  "communities": ["COM_XXXXXX"],
  "karma": 0,
  "bio": "string",
  "avatar": "https://res.cloudinary.com/...",
  "createdAt": "...",
  "updatedAt": "..."
}
```

Communities

GET /api/community

Public. All communities, sorted by memberCount descending.

200 returns { success, message, total, data: [<community>, ...] }

GET /api/community/:name

Public. Fetch one community by its name field (not communityId).

200 returns { success, message, data: <community> } **404:** not found.

POST /api/community (protected)

Create a community. The creator automatically becomes the first moderator.

Body

```
{ "name": "string", "description": "string" }
```

201 returns { success: true, message, community: <community> } **400:** missing name, or a community with that name already exists.

POST /api/community/:name/join (protected)

Join a community. Increments memberCount and adds the community's communityId to the user's communities array.

200 returns { success: true, message: "Successfully joined r/<name>" } **404:** community not found.

POST /api/community/:name/leave (protected)

Same as join, in reverse.

200 returns { success: true, message: "Successfully left r/<name>" }

POST /api/community/:name/banner (protected)

Upload a community banner image.

Body: multipart/form-data, field name image.

200 returns { success: true, message: "Banner updated", banner: "<cloudinary url>" } **400:** no image provided.

Community object shape

```
{
  "communityId": "COM_XXXXXX",
  "name": "string",
  "description": "string",
  "createdBy": "username",
  "moderators": ["username"],
  "memberCount": 1,
  "avatar": null,
  "banner": null,
  "rules": [{ "title": "string", "description": "string" }],
  "createdAt": "...",
  "updatedAt": "..."
}
```

Posts

GET /api/post

Public. List posts with filtering, search, sort, and pagination.

Query params (all optional):

Param	Type	Notes
communityId	string	filter to one community
search	string	full-text search on title and body
sort	newest or oldest	defaults to newest
page	number	defaults to 1
limit	number	defaults to 20

200 Response

```
{
  "success": true,
  "message": "All posts fetched successfully",
  "total": 42,
  "page": 1,
  "data": [ "<post>", "..." ]
}
```

GET /api/post/:postId

Public. Fetch a single post by postId.

200 returns { success, message, data: <post> } **404:** not found.

POST /api/post (protected)

Create a post. Optionally attach one image.

Use multipart/form-data if including an image; plain JSON works too since the route doesn't require the file.

```
title: string (required)
body: string
communityId: string (required)
image: file (optional, field name "image")
```

201 returns { success: true, data: <post> } **400**: missing title or communityId.

DELETE /api/post/:postId (protected)

Only the post's author can delete it.

200 returns { success: true, message: "Post deleted successfully" }
403: not the author. **404**: post not found.

Post object shape

```
{
  "postId": "POST_XXXXXX",
  "title": "string",
  "body": "string",
  "image": "https://res.cloudinary.com/... or null",
  "author": "username",
  "communityId": "COM_XXXXXX",
  "voteCount": 0,
  "commentCount": 0,
  "createdAt": "...",
  "updatedAt": "..."
}
```

Comments

GET /api/comment/:postId

Public. All comments for a post, oldest first. This is a flat list, not nested; use parentId on the frontend to build a tree.

200 returns { success, message, data: [<comment>, ...] }

POST /api/comment (protected)

```
{
  "postId": "string (required)",
  "body": "string (required)",
  "parentId": "string (optional, for replies)"
}
```

Also increments the parent post's commentCount.

201 returns { success: true, message, data: <comment> } **400**: missing body. **404**: missing or invalid postId.

DELETE /api/comment/:commentId (protected)

Only the comment's author can delete it. Decrements the post's commentCount.

200 returns { success: true, message: "Comment deleted successfully" } **403**: not the author. **404**: not found.

Comment object shape

```
{
  "commentId": "CMT_XXXXXX",
  "postId": "POST_XXXXXX",
  "parentCommentId": "CMT_XXXXXX or null",
  "body": "string",
  "author": "username",
  "voteCount": 0,
  "createdAt": "...",
  "updatedAt": "..."
}
```

Votes

POST /api/vote (protected)

Upvote or downvote a post or comment. Calling it again with the same value removes the vote (toggle); calling it with the opposite value flips it.

Body

```
{
  "targetId": "POST_XXXXXX or CMT_XXXXXX",
  "targetType": "post | comment",
  "value": 1
}
```

value must be 1 (upvote) or -1 (downvote).

200 Response

```
{ "success": true, "message": "Vote cast successfully", "voteCount":
5 }
```

400: missing fields, invalid value, or invalid targetType. **404:** target post/comment not found.

There's no separate endpoint to check the user's current vote on something. The frontend needs to track that locally, for example from the last response, if you want filled versus outline arrow icons.

File Uploads

Handled by Multer plus Cloudinary (middleware/upload.middleware.js). Applies to: - POST /api/post (field: image) - POST /api/community/:name/banner (field: image) - POST /api/users/me/avatar (field: image)

Rules: - Allowed formats: jpeg, jpg, png, webp - Max size: UPLOAD_LIMIT_MB env var, defaults to 5MB - Send these as FormData on the frontend, not JSON:

```
const formData = new FormData();
formData.append('image', file);
await api.post('/community/myname/banner', formData, {
  headers: { 'Content-Type': 'multipart/form-data' }
});
```

Response Shape and Error Handling

Most endpoints follow this shape:

```
{ "success": true, "message": "...", "data": {} }
```

Errors:

```
{ "success": false, "message": "..."} 
```

Status codes used across the API:

Code	Meaning
200	success
201	resource created
400	bad request or validation error
401	not authenticated (missing or invalid token)
403	authenticated but not authorized (for example, not the author)
404	resource not found
500	server error

Auth header on protected routes:

Authorization: Bearer <token>

A missing or invalid token returns 401.

CORS: the backend only allows requests from `http://localhost:5173` and the deployed Vercel URL by default (see `server.js` and the `CORS_ORIGIN` env var). If your frontend runs on a different origin locally, update the allowed origins list or the `CORS_ORIGIN` env var.

Known Issues

Worth flagging to whoever owns the backend:

1. **No pagination on GET /api/users.** It returns every user in the database at once. Fine for a small dataset, but worth knowing if the user list grows.
2. **No endpoint to fetch the current user's own vote state** on a post or comment, so vote button UI (filled versus outline arrow) needs to be derived and cached client-side.
3. **PUT /api/users/me doesn't pre-check username uniqueness** before attempting the update. A duplicate username returns a raw 500 instead of a clean 400 with a friendly message.